

Session Hijacking

Demonstracao local de sequestro de sessao por reutilizacao de cookie e mitigacao com atributos seguros, expiracao e invalidacao no servidor.

Tema

05 - Session Hijacking

Integrantes

Integrante 1
Integrante 2

Local

Dados ficticios

Express + express-session

Antes e depois

Objetivo e escopo etico

- Provar que uma sessao vulneravel pode ser reutilizada em outro cliente local.
- Mostrar que a correcao reduz exposicao e bloqueia cookies invalidos, antigos, expirados ou apos logout.
- Comparar codigo vulneravel e codigo corrigido de forma didatica.

Limite da demonstracao

Somente `127.0.0.1`/localhost, usuarios fake (`alice`, `bruno`) e dados ficticios. Nada de contas reais, alvos externos, sniffing, XSS, CSRF ou tunel publico.

Arquitetura do laboratorio

Navegador

Login local e cookie de sessao enviado a cada request.



Express

``/login`, `/dashboard`, `POST /logout` e `requireAuth`.`



express-session

Cookie guarda o ID; dados de sessao ficam no servidor.

Modo vulneravel

``sid`` inseguro para provar o ataque.

Modo corrigido

``__Host-session`, `HttpOnly`, `Secure`, `SameSite`, `maxAge`.`

Prova

``npm test`` e demo manual com DevTools/cURL.

Como acontece o Session Hijacking

1. Usuario legitimo autentica e recebe cookie de sessao.
2. Outro cliente obtem o mesmo identificador de sessao.
3. Servidor aceita o cookie e associa o segundo cliente a sessao existente.

Ideia central

Um cookie bearer, ou identificador de sessao ativo, funciona como credencial. Quem apresenta o cookie valido recebe a sessao enquanto ela existir.

O objetivo da mitigacao e reduzir exposicao, limitar a janela de uso e invalidar a sessao no servidor quando necessario.

Codigo vulneravel: cookie `sid` reutilizavel

src/session/vulnerable-session.js

```
session({
  name: "sid",
  resave: false,
  saveUninitialized: false,
  cookie: {
    httpOnly: false,
    secure: false,
    sameSite: false,
    maxAge: 24 * 60 * 60 * 1000
  }
});
```

Por que e vulneravel?

Vulnerabilidade demonstrada: reutilizacao de cookie de sessao sem nova autenticao.

- `httpOnly: false` aumenta exposicao no cliente.
- `secure: false` permite envio sem HTTPS.
- `sameSite: false` remove restricao cross-site.
- `maxAge` longo amplia a janela de reuso.

Demonstracao do ataque local

1. `npm run dev`.
2. Cliente A: login como `alice`.
3. Cliente B: `/dashboard` redireciona para `/login`.
4. Copiar `sid` do Cliente A e inserir no Cliente B.
5. Cliente B acessa `/dashboard` sem senha.

Fallback com cURL

```
curl.exe -i http://127.0.0.1:3000/dashboard ^  
-H "Cookie: sid=<copied-sid-value>"
```

Nos slides ou screenshots, redact o valor real do cookie.

Impacto e principio violado

Impacto observado

- Segundo cliente acessa dados privados ficticios da Alice.
- Nenhuma senha e enviada no replay.
- O cookie passa a representar a identidade da sessao.

Principio violado

Controle insuficiente sobre identificador de sessao: o servidor confia no cookie apresentado e nao distingue o cliente legitimo do replay.

Evidencia automatizada: ``tests/session-reuse-attack.test.js`` prova que uma requisicao separada reutiliza ``sid`` e recebe dados fake da Alice.

Codigo corrigido: atributos seguros

src/session/fixed-session.js

```
session({
  name: "__Host-session",
  resave: false,
  saveUninitialized: false,
  cookie: {
    httpOnly: true,
    secure: true,
    sameSite: "strict",
    maxAge: 5 * 60 * 1000,
    path: "/"
  }
});
```

O que muda?

- `HttpOnly`: reduz exposicao a JavaScript.
- `Secure`: envio apenas via HTTPS no caminho seguro.
- `SameSite=Strict`: restringe envio cross-site.
- `maxAge` curto: diminui janela de abuso.
- `\_\_Host-session`: cookie mais restrito, com `Path=/` e sem `Domain`.

Lifecycle: login, expiracao e logout

Login

No modo corrigido, a sessao e regenerada antes de gravar `userId`.

Expiracao

`maxAge` de 5 minutos limita o tempo util do cookie copiado.

Logout

`POST /logout` executa `req.session.destroy`, limpa o cookie e redireciona para `/login`.

O HTTP local com `npm run dev:fixed` e fallback de inspecao. Protecao real do atributo `Secure` exige HTTPS; os testes validam o caminho seguro com headers de proxy HTTPS.

Verificacao: antes funciona, depois falha

Antes

Replay do `sid` vulneravel chega ao `/dashboard` e mostra `Alice Demo` / `LAB-ALICE-001`.

Depois

Cookie antigo, invalido, expirado ou apos logout retorna `302 Location: /login`.

```
npm test
```

```
mitigation verification:
```

- vulnerable sid replay reaches dashboard
- obsolete vulnerable sid is rejected in fixed mode
- copied fixed cookie fails after logout

OWASP e Secure SDLC

- OWASP Top 10 2025: A07 Authentication Failures.
- OWASP 2021: A07 Identification and Authentication Failures.
- OWASP Session Management: proteger o ID de sessao, expirar e invalidar.
- Secure SDLC: identificar, explorar de forma controlada, corrigir e verificar.

Mensagem honesta

As mitigacoes reduzem a chance e a janela de sequestro. Um cookie ativo ainda e sensivel ate expirar ou ser invalidado no servidor.

Conclusao e referencias

Conclusao

O lab demonstrou, localmente, que um cookie de sessao inseguro pode ser reutilizado. Depois das correcoes, sessoes invalidas ou encerradas nao abrem mais o dashboard protegido.

Referencias

- OWASP Top 10 A07 Authentication Failures.
- OWASP Session Management Cheat Sheet.
- MDN Set-Cookie.
- Express `express-session`.
- Codigo e testes deste laboratorio.